

MODULE 1

Introduction to NoSQL Databases

Why NoSQL databases exist, how the four NoSQL models differ, and where MongoDB's document model fits.

Why NoSQL

Four NoSQL models

**MongoDB
documents**

**MongoDB
architecture**

**When MongoDB
fits**

LEARNING OBJECTIVES

What you will understand by the end of this module.

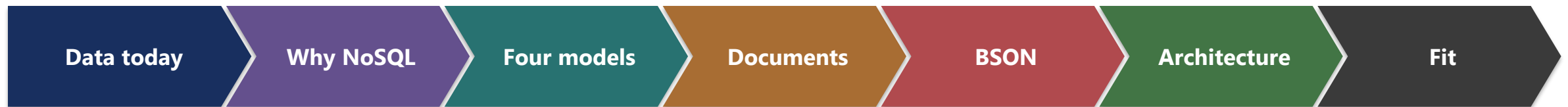


BY THE END OF THIS MODULE, YOU CAN

1. Define NoSQL as a category, not one product
2. Compare the four NoSQL data models
3. Explain documents, collections and BSON
4. Describe the basic MongoDB architecture
5. Judge when MongoDB is — and isn't — a fit
6. Explore training_store in mongosh



MODULE ROADMAP



SIX QUESTIONS THIS MODULE ANSWERS

WHY

did NoSQL databases appear?

WHAT

are the four NoSQL models?

WHAT

exactly is a MongoDB document?

HOW

is MongoDB built?

WHEN

is MongoDB the right choice?

HOW

do we read real data in mongosh?

KEY TAKEAWAY Choose a data model from the access pattern — not from a product's popularity.

KEY TERMS FOR THIS MODULE

Plain definitions for the words we'll use most.

1 NoSQL

Not Only SQL: databases that store data in models other than relational tables.

2 Document

One record stored as field-value pairs, like a JSON object — one product or one order.

3 Collection

A group of related documents inside a database, such as products or orders.

4 BSON

Binary JSON: MongoDB's typed storage format, with dates, decimals and ObjectIds.

5 _id and ObjectId

_id uniquely identifies a document in its collection; ObjectId is its default value.

6 Flexible schema

Documents in one collection may have different fields; validation rules are optional.

7 Access pattern

How the application reads and writes its data — it drives the model you choose.

8 Replica set

Several mongod servers holding copies of the same data, for high availability.

KEY TAKEAWAY Agree on the words first — every later slide builds on them.

FULL FORMS USED IN THIS MODULE

Every abbreviation on these slides, spelled out.

ACID	Atomicity, Consistency, Isolation, Durability	ID	Identifier	RAM	Random Access Memory
API	Application Programming Interface	IoT	Internet of Things	SKU	Stock Keeping Unit
BSON	Binary JSON	ISBN	International Standard Book Number	SQL	Structured Query Language
CPU	Central Processing Unit	JSON	JavaScript Object Notation	USB	Universal Serial Bus
CRUD	Create, Read, Update, Delete	NoSQL	Not Only SQL		
FK	Foreign Key	PK	Primary Key		

KEY TAKEAWAY When an abbreviation isn't clear, the full form is on this slide.

ONE STORY THROUGH THE MODULE

The training_store sample database — and the exercises you will run on it.



THE TRAINING_STORE SAMPLE DATABASE

customers
6 documents

products
13 documents

orders
17 documents

reviews
6 documents



ONE ORDER, STEP BY STEP

Luis (C204)
shops

→ Adds shoe +
keyboard

→ Order O6102
placed

→ Payment
PAID

→ Order
DELIVERED

THE MODULE'S QUESTION Where should data like this live — and in what shape should each record be stored?

KEY TAKEAWAY The same store runs through every slide, so each new idea builds on the last.

YOUR LAB: TRAINING_STORE

Exercise 1.3 opens this database in mongosh. The instructor loads it before class; Module 2 shows you how.

FIRST COMMANDS

```
show dbs
use training_store
show collections
db.orders.findOne()
```

PART 1: WHY NOSQL

Start with the data modern applications produce.

Part 1
Why NoSQL

Part 2
Four NoSQL models

Part 3
MongoDB's data model

Part 4
Architecture and fit

SO FAR

- ✓ You know the goals of this module
- ✓ You met the training_store database

NEXT QUESTION

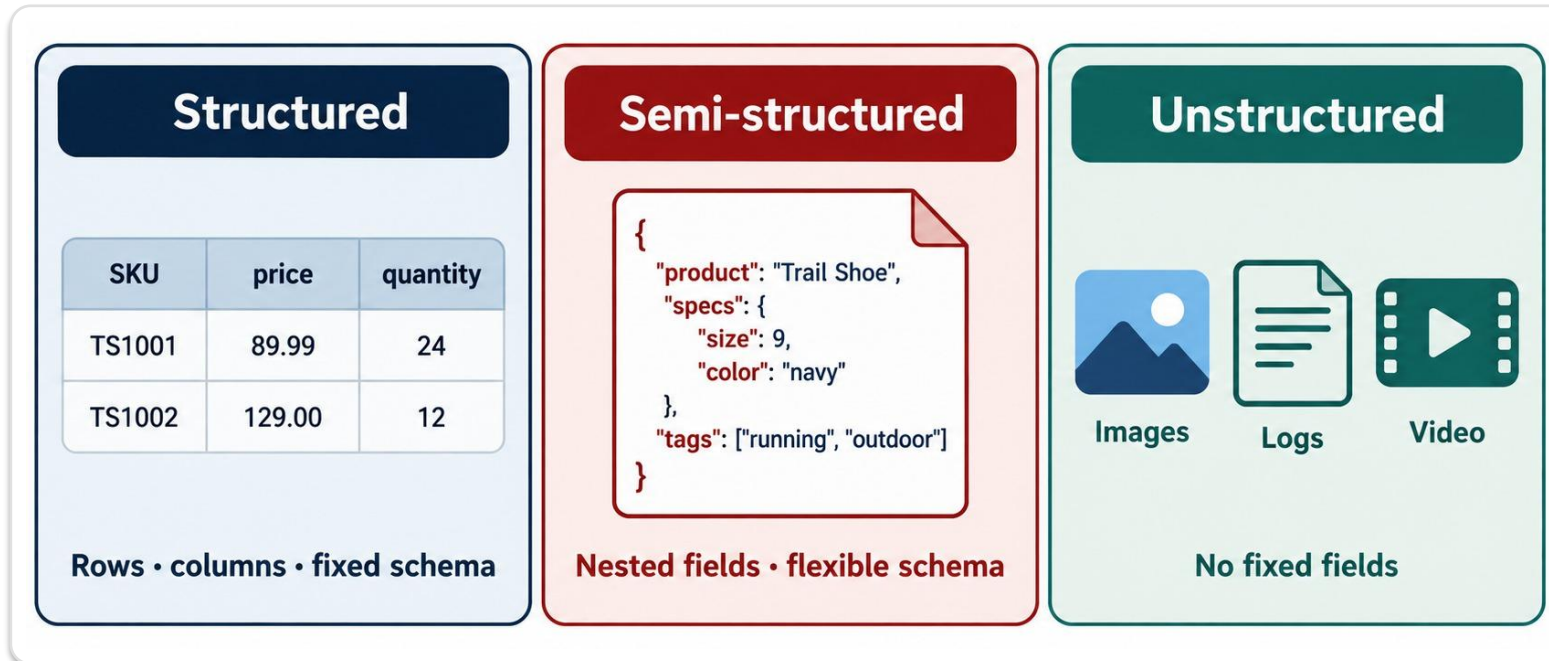
Why did databases beyond relational tables become necessary?

IN THIS PART

- 1 Three kinds of application data
- 2 Where relational databases work well
- 3 Where they start to struggle
- 4 What NoSQL means
- 5 Why organizations adopt it

STRUCTURED, SEMI-STRUCTURED AND UNSTRUCTURED DATA

Rows and columns, nested fields — and data with no fixed fields.



Structured

Rows and columns with a fixed schema — prices and quantities.

Semi-structured

Nested fields and arrays whose shape can vary — product specs and tags.

Unstructured

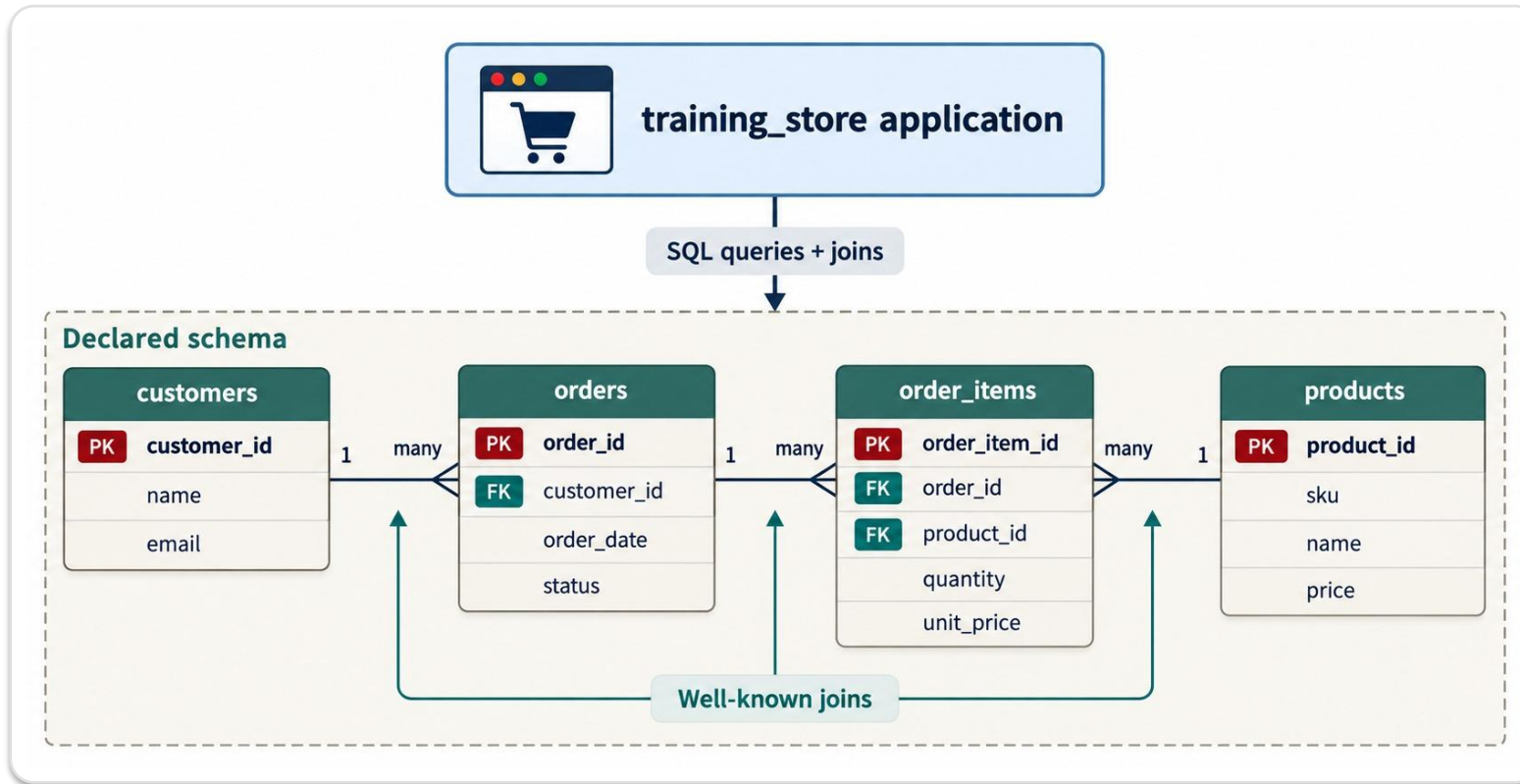
No fixed fields at all — images, logs and video.

OUR DATASET training_store products are semi-structured: each category has its own attributes.

KEY TAKEAWAY Tables are built for structured data — much of what modern apps store is semi-structured.

WHERE RELATIONAL DATABASES WORK WELL

A declared schema, enforced keys and well-known joins.



Declared schema

Tables, columns and types are fixed before data arrives.

Enforced relationships

Foreign keys tie every order to a real customer.

Well-known joins

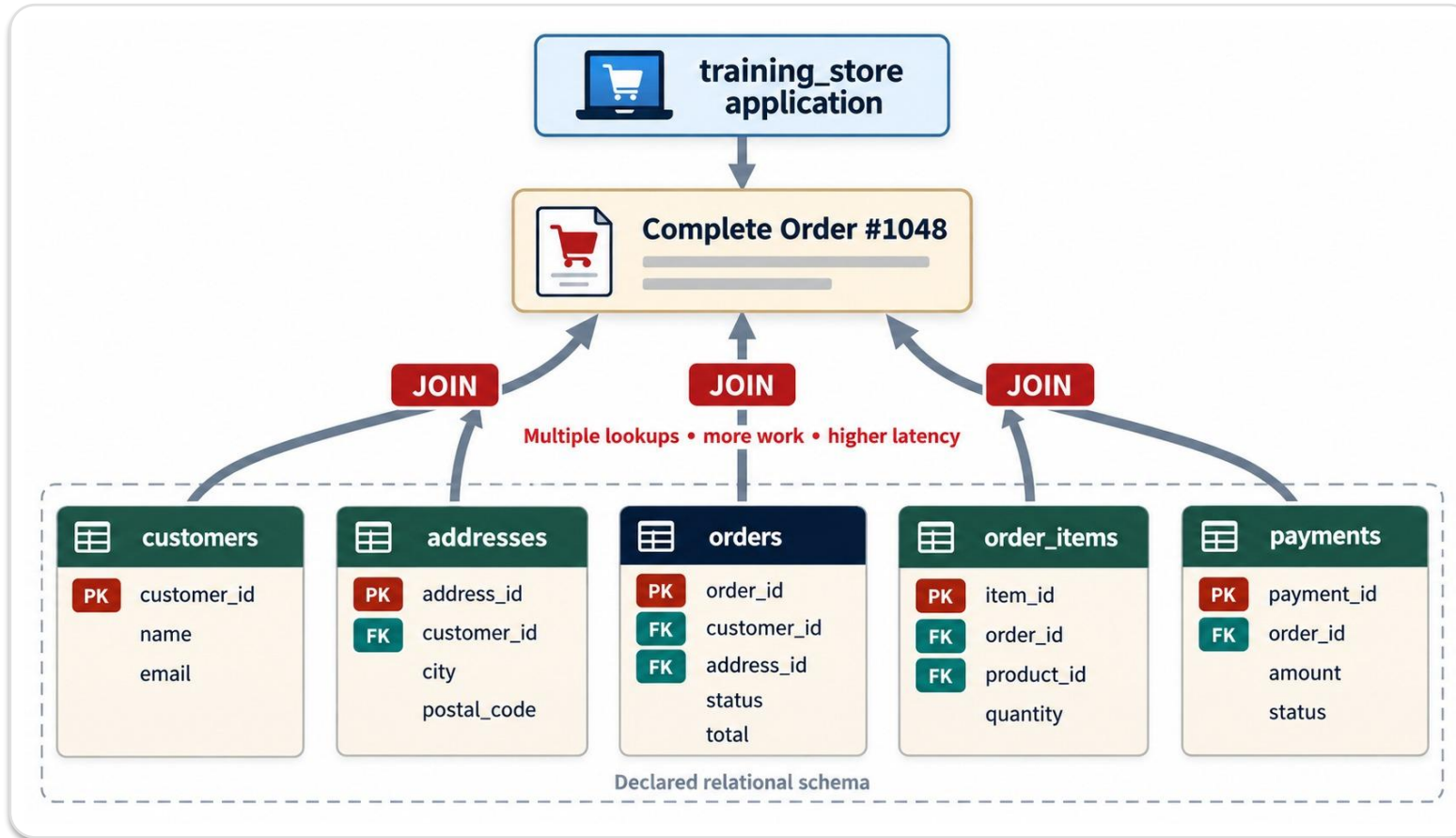
SQL rebuilds the full picture from customers, orders, items and products.

STILL THE RIGHT TOOL Stable structure plus strict integrity.

KEY TAKEAWAY When the structure is stable and relationships must be enforced, relational databases work very well.

CHALLENGES WITH TRADITIONAL RELATIONAL MODELS

One complete order means joining five tables.



Many lookups

Customer, address, order, items and payment — joined on every read.

Rigid change

A new product attribute needs a migration or more optional columns.

Harder to scale out

Joined tables are hard to spread across many servers.

COSTS, NOT FAILURES They grow with change and volume.

KEY TAKEAWAY The more pieces one business object is split into, the more work every read and every change does.

ADDING A NEW KIND OF PRODUCT

The catalog starts selling shoes — what has to change?

Relational: change the table first

```
ALTER TABLE products
  ADD COLUMN sizes    VARCHAR(40);
ALTER TABLE products
  ADD COLUMN color    VARCHAR(20);
ALTER TABLE products
  ADD COLUMN material VARCHAR(20);
-- laptops and books now carry
-- three empty columns each
```

Relational

Every new attribute is a schema change, plus empty columns elsewhere.

✓ Document: insert the new shape

```
db.products.insertOne({
  sku: "S200",
  name: "Running Shoe",
  category: "SHOE",
  price: Decimal128("129.99"),
  attributes: {
    sizes: [7, 8, 9, 10],
    color: "Blue", material: "Mesh"
  }
})
```

Document

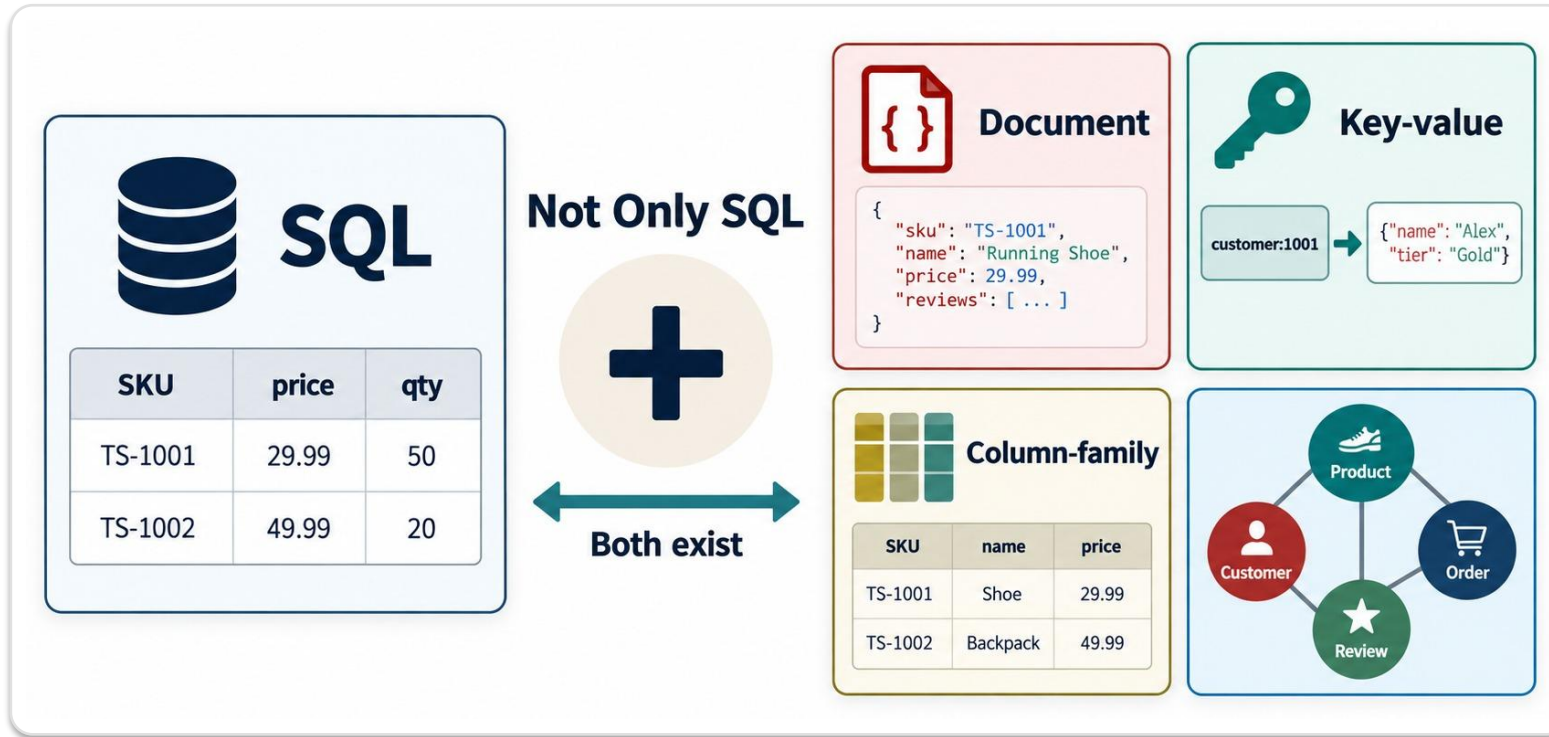
Each product carries only the fields it needs.

STILL DESIGNED Flexible isn't random —
Module 3 models it.

KEY TAKEAWAY A fixed schema turns every new attribute into a migration; documents let each product carry its own fields.

WHAT DOES NOSQL MEAN?

Not Only SQL — relational and non-relational models side by side.



Not Only SQL

Databases that store data in models other than relational tables.

Four families

Document, key-value, column-family and graph.

BOTH EXIST Many real systems use SQL and NoSQL together.

KEY TAKEAWAY NoSQL adds data models — it doesn't make SQL databases obsolete.

WHY ORGANIZATIONS ADOPT NOSQL

Application growth creates four needs.



Flexible schema

Fields change as the product changes — no migration first.

Horizontal scale

Add servers instead of buying a bigger one.

Availability

Copies on several servers survive one failure.

Faster delivery

Store objects the way the application already uses them.

KEY TAKEAWAY The right database depends on the data, the access patterns, the consistency needs and the scale.

EXERCISE: WHAT DOES ONE ORDER PAGE COST?

Count the work behind a single page in the relational design.

PRACTICE EXERCISE

10 MIN

SCENARIO

The order page shows O6102: Luis Romero's Running Shoe and Wireless Keyboard, total 211.38.

In the relational design on the challenges slide, that order lives in five tables.

YOUR TASKS

- 1 List the tables the page must read
- 2 Count the joins needed
- 3 Name the key columns that link them
- 4 Sketch what one order document would hold

EXPECTED OUTCOME

- ✓ customers, addresses, orders, order_items, payments
- ✓ Four joins — five with products for item names
- ✓ customer_id, address_id, order_id, product_id
- ✓ One document: order, items and shipping address

KEY TAKEAWAY One page can hide many joins — a document shaped like the page needs one read.

PART 2: FOUR NOSQL MODELS

NoSQL is a family. Meet its four members.

✓ Part 1
Why NoSQL

Part 2
Four NoSQL models

Part 3
MongoDB's data model

Part 4
Architecture and fit

SO FAR

- ✓ Data comes structured, semi-structured and unstructured
- ✓ Relational tables excel at stable, related data
- ✓ NoSQL adds models for flexibility and scale

NEXT QUESTION

How do the four NoSQL models store data differently — and what is each one best at?

IN THIS PART

- 1 The four families at a glance
- 2 Document databases
- 3 Key-value databases
- 4 Column-family databases
- 5 Graph databases
- 6 All five models side by side

THE FOUR MAJOR NOSQL DATABASE TYPES

Document, key-value, column-family and graph.



Document

JSON-like aggregates

```
{
  "product": "Trail Shoe",
  "specs": { "size": 9 },
  "tags": ["running"]
}
```



Key-Value

Opaque value by key

"cart:1001"



Column-family

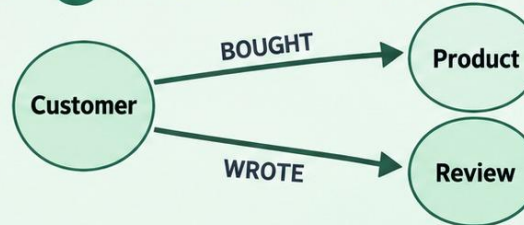
Wide rows • variable columns

product	price	stock_CA	stock_US
TS1001	89.99	24	41



Graph

Nodes + relationships



Document

Whole records as JSON-like documents — MongoDB, Couchbase.

Key-value

One opaque value per key — Redis, Amazon DynamoDB.

Column-family

Wide rows with variable columns — Apache Cassandra, HBase.

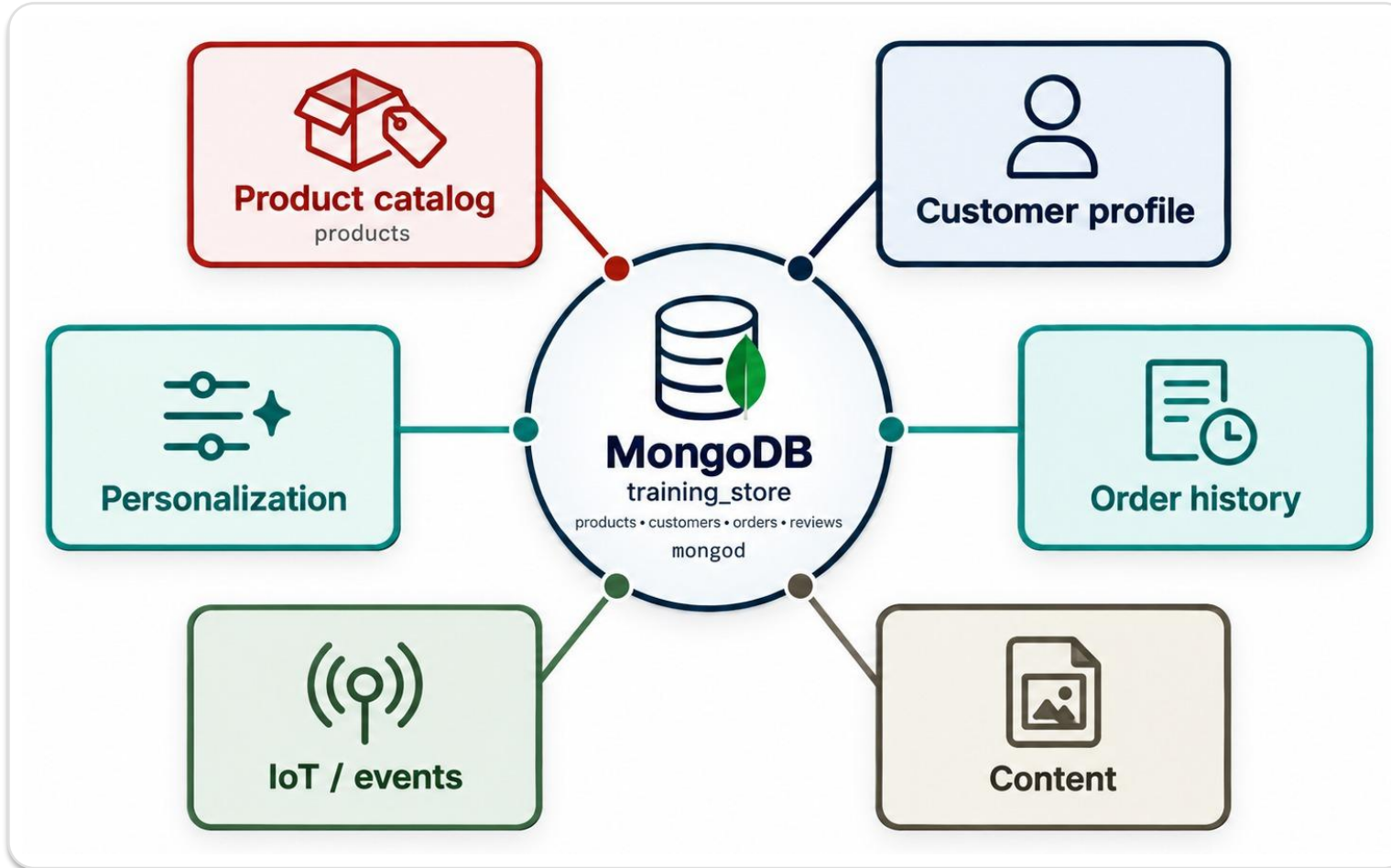
Graph

Nodes and relationships — Neo4j, Amazon Neptune.

KEY TAKEAWAY Each NoSQL family is optimized for a different way of reading and writing data.

DOCUMENT DATABASES

Store one application object as one self-contained document.



A training_store product (trimmed)

```
{ sku: "S210",  
  name: "Trail Shoe",  
  category: "SHOE",  
  price: Decimal128("89.99"),  
  attributes: {  
    sizes: [8, 9, 10, 11],  
    color: "Grey",  
    material: "Synthetic" },  
  tags: ["trail", "sports"] }
```

Good fit

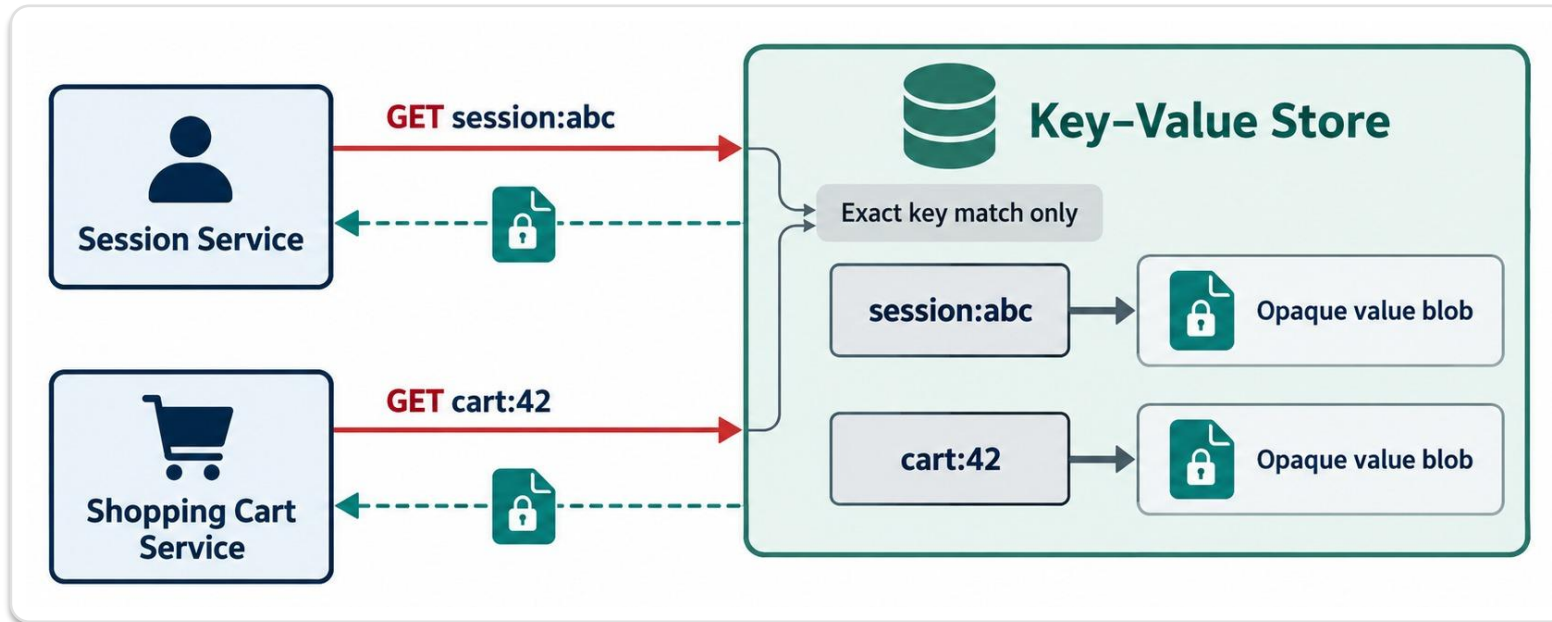
Catalogs, customer profiles, content and order management.

MAIN ADVANTAGE One object stays one document — not many tables.

KEY TAKEAWAY A document database keeps each business object together, in the shape the application uses.

KEY-VALUE DATABASES

The application supplies the exact key; the store returns the value.



How it works

GET session:abc returns that session's value — nothing else.

Good fit

Caching, sessions, shopping carts, preferences, counters.

Trade-off

Very fast by key; weak for queries about what's inside the value.

EXAMPLES

Redis

Amazon DynamoDB

KEY TAKEAWAY Key-value stores are the fastest path when the application always knows the key.

COLUMN-FAMILY DATABASES

Rows found by row key, with related columns grouped into families.

Lookup by row key

 Row key	 profile		 orders		 activity	
	name	tier	latest	count	lastSeen	views
customerId: 101	Ava	Gold	ORD-901	4	Sep 18	12
customerId: 205	Liam	—	ORD-944	1	—	3
customerId: 318	Noor	Silver	—	—	Sep 20	8

Row key

Every lookup starts from the row key, e.g. customerId 101.

Column families

Related columns sit together: profile, orders, activity.

Sparse rows

A row stores only the columns it has — gaps cost nothing.

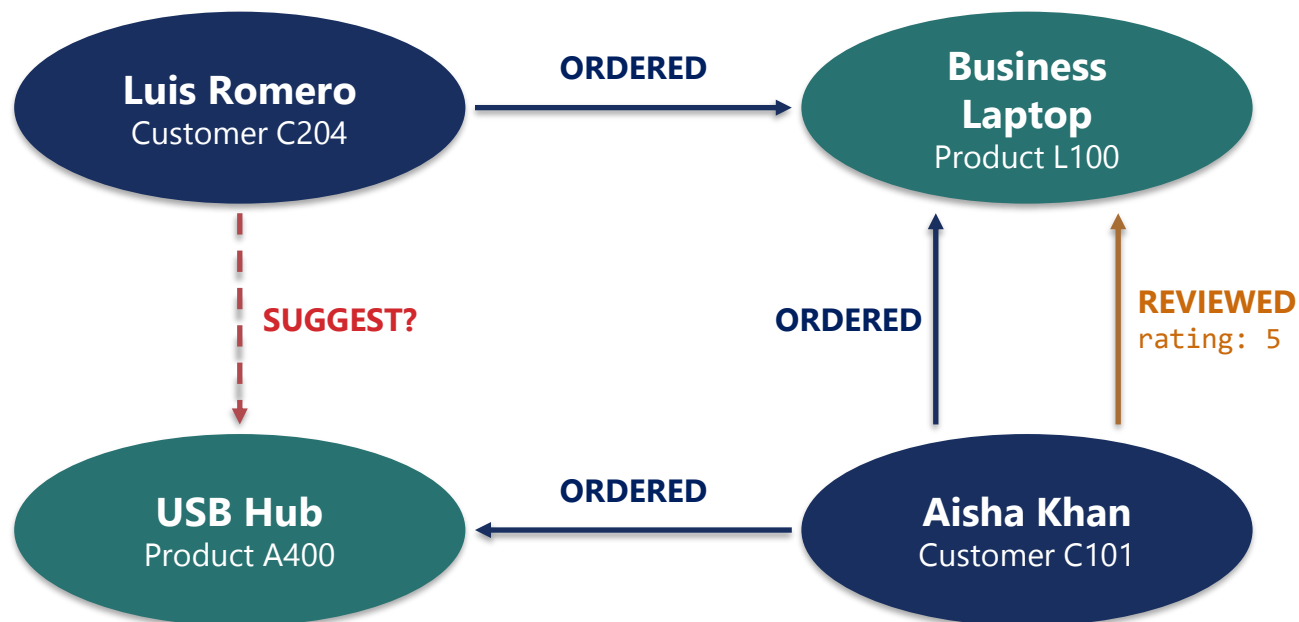
Good fit

Huge write volumes: events, sensors, time series — Cassandra, HBase.

KEY TAKEAWAY Column-family stores handle huge, distributed write volumes looked up by row key.

GRAPH DATABASES

Nodes, relationships and properties — real training_store facts drawn as a graph.



Nodes and edges

Nodes are things, edges are relationships — and both can carry properties.

Good fit

Social networks, recommendations, fraud detection and access relationships.

EXAMPLES

Neo4j

Amazon Neptune

TRAVERSAL Luis → L100 ← Aisha → A400: suggest the USB Hub to Luis.

STRENGTH Multi-step walks along relationships stay fast.

KEY TAKEAWAY Choose a graph database when the relationships matter as much as the things they connect.

COMPARING THE DATA MODELS

The four NoSQL families — and relational — side by side.

Model	Data model	Main strength	Typical use
Relational	Tables, fixed schema	Integrity, joins, transactions	Ledgers, finance
Document	JSON-like documents	Flexible, app-friendly records	Catalogs, profiles, orders
Key-value	A key mapped to a value	Extremely fast lookup	Caching, sessions, carts
Column-family	Wide rows by row key	Huge distributed writes	Events, time series
Graph	Nodes and relationships	Relationship traversal	Fraud, recommendations

More than one can work

Many workloads have two workable models — justify your first choice.

Look at the access

By key? Whole object? Huge writes? Deep relationships?

EXERCISE 1.1 You'll match six workloads to these five models.

KEY TAKEAWAY Start from how the data is read and written — then pick the model built for that access pattern.

PART 3: MONGODB'S DATA MODEL

From the document family in general to MongoDB in particular.



SO FAR

- ✓ Four NoSQL families, each with its strength
- ✓ Choose the model from the access pattern
- ✓ Document databases keep whole objects together

NEXT QUESTION

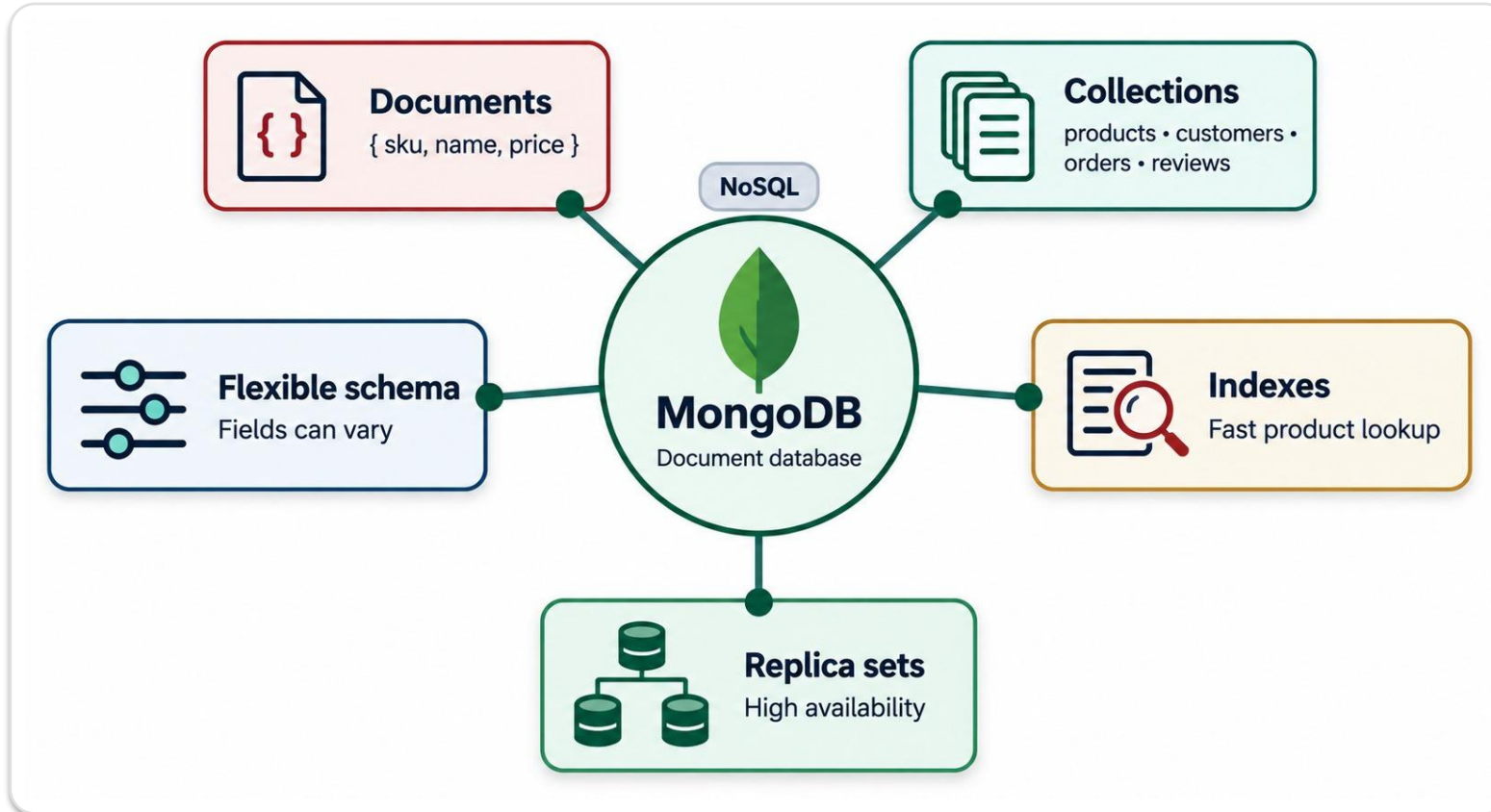
What exactly is a MongoDB document — and how is the data organized?

IN THIS PART

- 1 MongoDB at a glance
- 2 Database, collection, document, field
- 3 Relational terms in MongoDB
- 4 BSON types in a document
- 5 From rows to documents
- 6 Embed or reference
- 7 Flexible schema in one collection

INTRODUCING MONGODB

A document database that stores BSON documents in collections.



Document database

Data is stored as documents, grouped into collections.

BSON

Binary JSON — adds types JSON lacks: dates, decimals, ObjectIds.

training_store

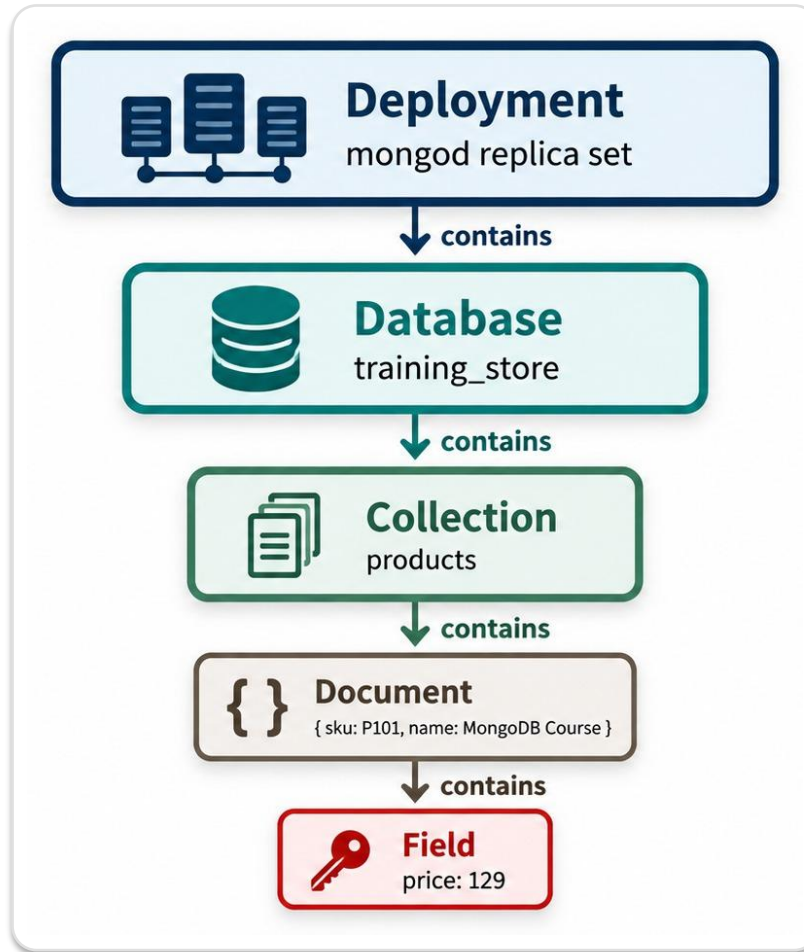
products, customers, orders and reviews — our four collections.

BEYOND STORAGE Indexes and replica sets are built in.

KEY TAKEAWAY MongoDB is a document database: BSON documents, grouped into collections, with indexing and replication built in.

THE MONGODB DATA HIERARCHY

Deployment → *database* → *collection* → *document* → *field*.



Walk it in mongosh

```
show dbs
use training_store
show collections
db.products.findOne()
```

Created on first write

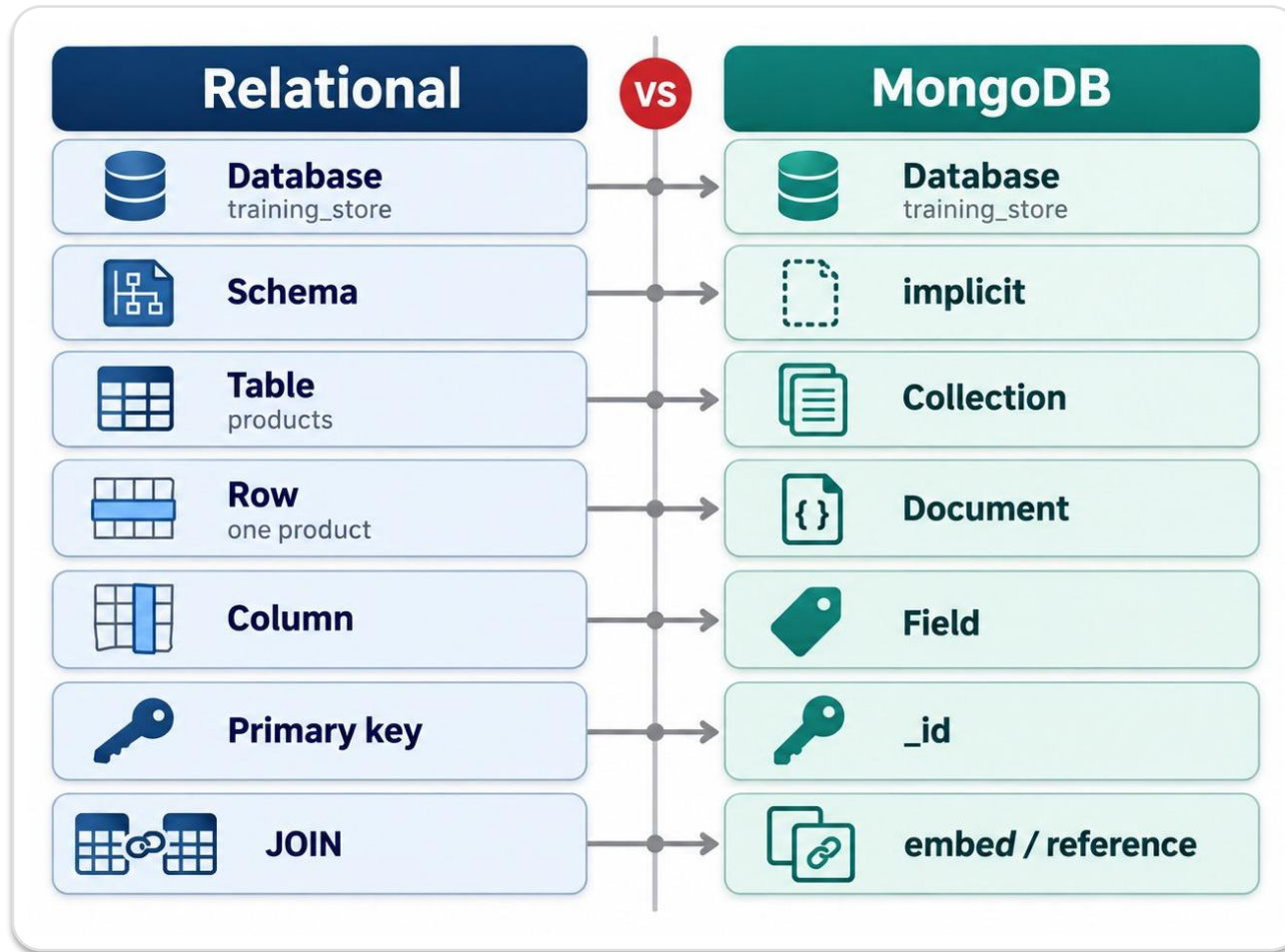
A database or collection appears the first time you insert into it.

OUR DATASET 13 products — e.g. B300, MongoDB Fundamentals.

KEY TAKEAWAY Every mongosh command works at one level of the hierarchy: database, collection, document or field.

RELATIONAL-TO-MONGODB TERMINOLOGY

The words map neatly — the designs often don't.



Row → document

A document can hold nested objects and arrays a row can't.

JOIN → embed / reference

Embed what is read together; reference the rest. \$lookup joins when needed.

Schema → implicit

Fields come from the documents; add validation rules when needed.

NOT ONE-TO-ONE Don't copy each table into a collection.

KEY TAKEAWAY Table, row and column become collection, document and field — but one document often replaces several joined rows.

ANATOMY OF A MONGODB DOCUMENT

Every field keeps its BSON type.



_id

Unique in its collection; an ObjectId is added if you don't supply one.

Typed values

String, Decimal128, Boolean, Date, Array and embedded document.

BSON

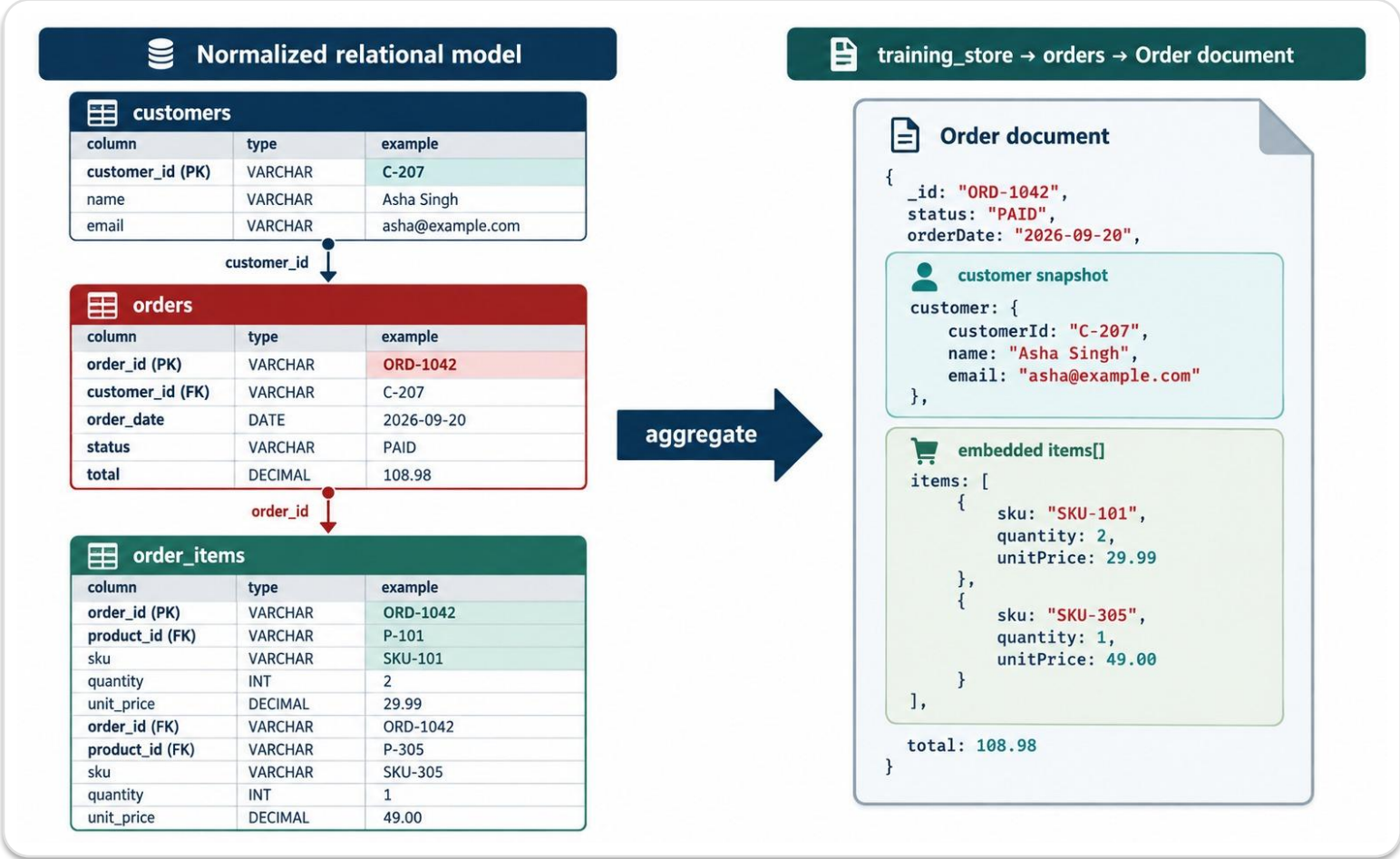
Stored as typed binary, shown as JSON-like text in mongosh.

MONEY Store prices as Decimal128, never as strings.

KEY TAKEAWAY BSON keeps real types, so IDs, money and dates behave correctly in queries.

FROM ROWS TO DOCUMENTS

Three normalized tables become one order document.



EMBED OR REFERENCE? HOW TRAINING_STORE DECIDES

Real documents from the orders and reviews collections.

✓ orders: O6102 (trimmed)

```
{
  orderNumber: "O6102",
  customerId: ObjectId("..."), // reference
  items: [ // embedded
    { sku: "S200", quantity: 1,
      unitPrice: Decimal128("129.99") },
    { sku: "P1001", quantity: 1,
      unitPrice: Decimal128("49.99") }
  ],
  shippingAddress: { city: "Austin", ... },
  total: Decimal128("211.38")
}
```

Embed

Items and the shipping address are read with the order and keep purchase-time values.

✓ reviews: their own collection (trimmed)

```
{
  productId: ObjectId("..."), // reference
  sku: "S200",
  customerId: ObjectId("..."), // reference
  rating: 4,
  title: "Comfortable",
  body: "Good for daily runs."
}
```

Reference

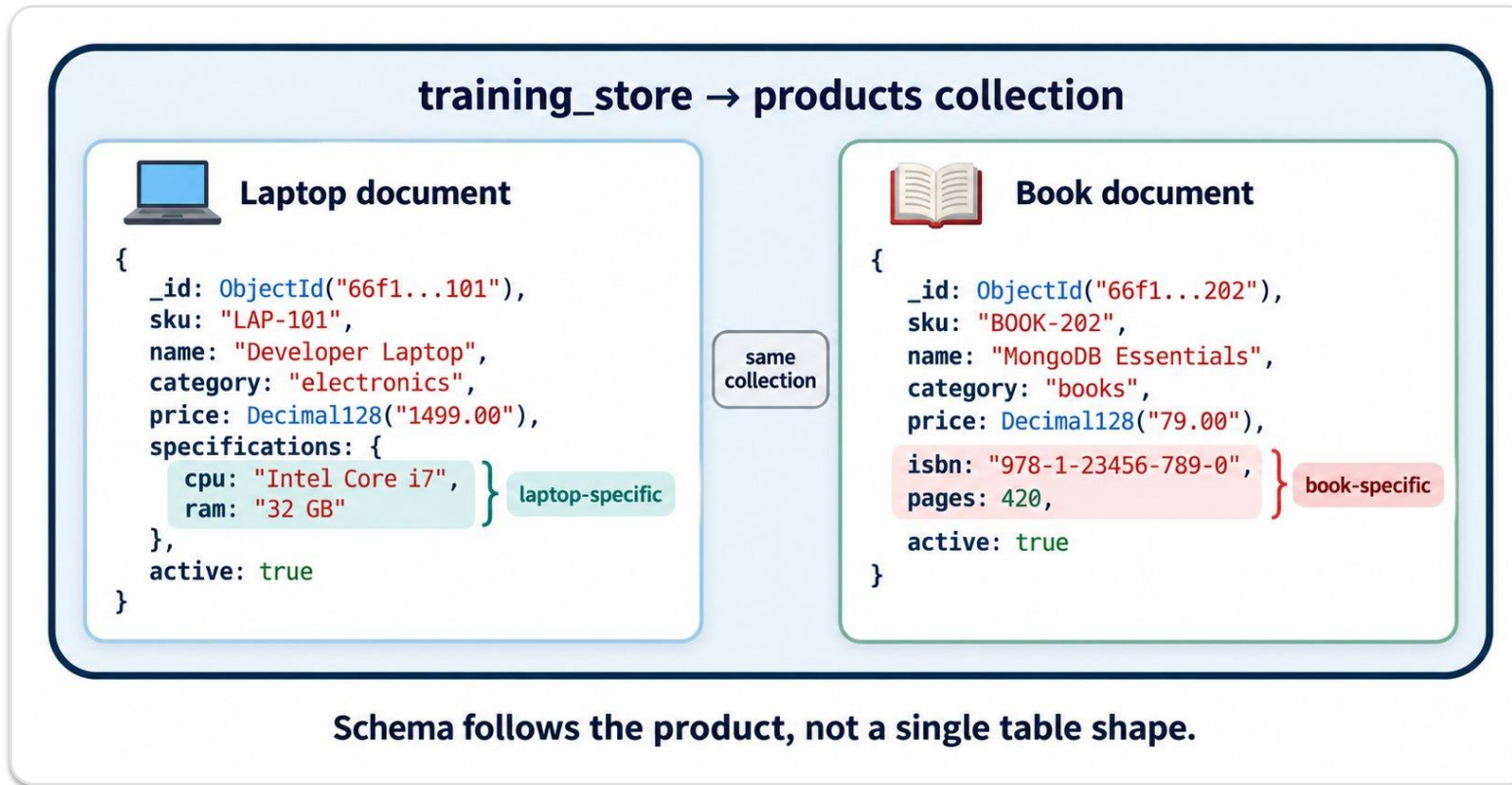
Customers are shared and change; reviews grow without limit.

RULE OF THUMB Store together what the app reads together.

KEY TAKEAWAY Embed what is read together and belongs to one object; reference what is shared, changes or grows without limit.

FLEXIBLE SCHEMA WITHIN A COLLECTION

Two products, one collection, different fields.



Different fields

The laptop has cpu and ram; the book has isbn and pages.

A shared core

Both keep `_id`, `sku`, `name`, `category`, `price` and `active`.

In training_store

LAPTOP, SHOE, BOOK and ACCESSORY each keep their own attributes.

FLEXIBLE ≠ UNPLANNED Schema validation can require key fields.

KEY TAKEAWAY Documents in one collection can differ — design the shared core, and validate what must always be there.

EXERCISE: TRANSLATE SQL THINKING INTO MONGODB

Rewrite a relational description of the store in MongoDB terms.

PRACTICE EXERCISE

10 MIN

SCENARIO

A teammate describes training_store in relational terms:

"The products table has 13 rows."
"Each row's key is product_id."
"Order lines live in order_items."
"We join them to show an order."

YOUR TASKS

- 1 Rewrite each sentence in MongoDB terms
- 2 Name the field that replaces product_id
- 3 Say what replaces the order_items join
- 4 Write the mongosh command that counts products

EXPECTED OUTCOME

- ✓ The products collection holds 13 documents
- ✓ _id — plus sku as a unique business key
- ✓ An items array embedded in each order
- ✓ `db.products.countDocuments()` returns 13

KEY TAKEAWAY Speaking MongoDB's vocabulary is the first step to thinking in documents.

PART 4: ARCHITECTURE AND FIT

How MongoDB runs — and when to choose it.

✓ Part 1
Why NoSQL

✓ Part 2
Four NoSQL models

✓ Part 3
MongoDB's data model

Part 4
Architecture and fit

SO FAR

- ✓ MongoDB stores BSON documents in collections
- ✓ Embed what is read together; reference the rest
- ✓ A flexible schema still needs design

NEXT QUESTION

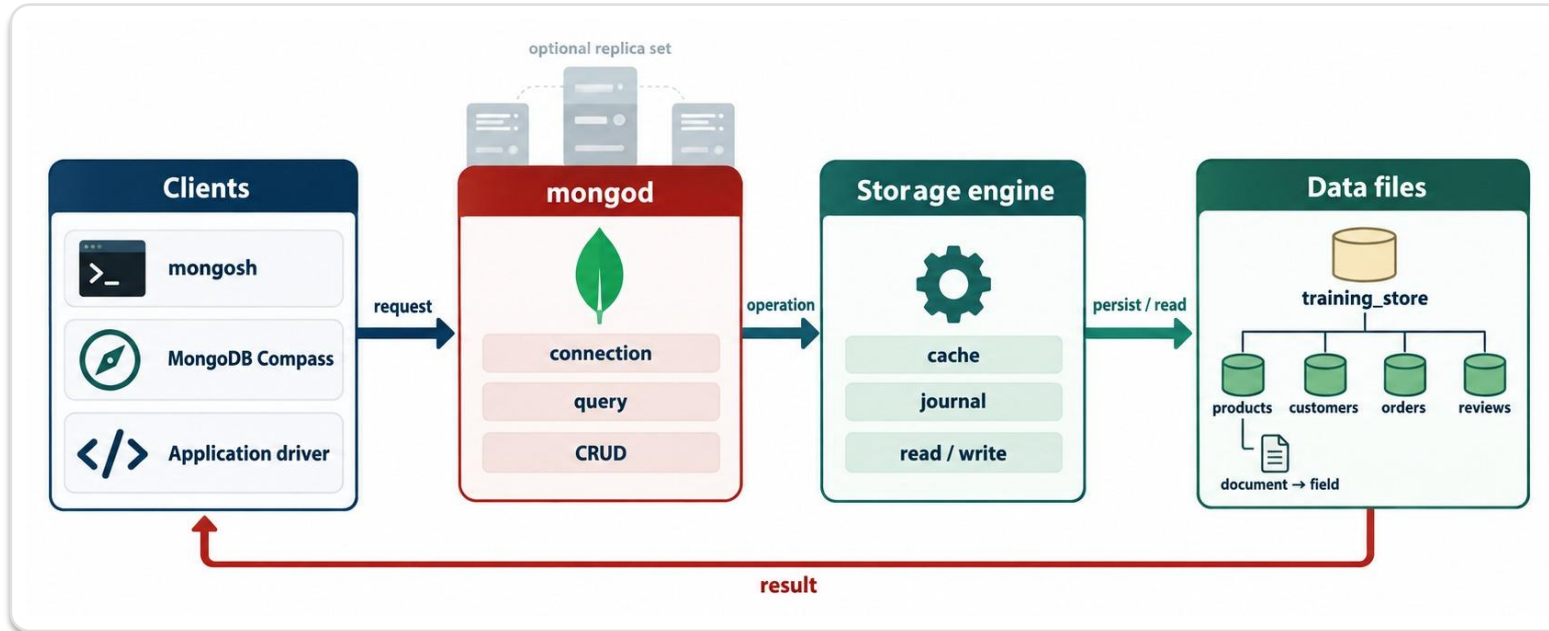
How does MongoDB run — and when is it the right choice?

IN THIS PART

- 1 Clients, mongod and the storage engine
- 2 How a request is processed
- 3 Replica sets and sharded clusters
- 4 Core features in mongosh
- 5 When MongoDB fits — and when it doesn't
- 6 A modernization scenario
- 7 Misconceptions and anti-patterns

MONGODB ARCHITECTURE OVERVIEW

Clients, the mongod server, the storage engine and the data files.



Clients

mongosh, MongoDB Compass and application drivers.

mongod

The server process: connections, queries, CRUD, indexes, authentication.

Storage engine

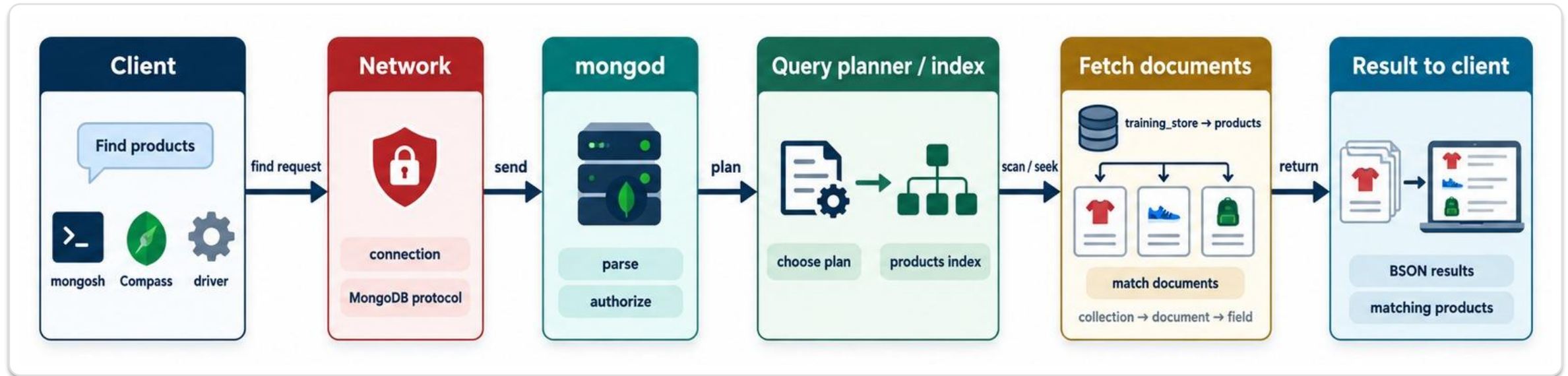
WiredTiger by default: cache, journal, reads and writes.

MODULE 2 You'll install mongod and connect with mongosh.

KEY TAKEAWAY Every client talks to mongod — the server process that owns storage, queries and indexes.

HOW A MONGODB REQUEST IS PROCESSED

Follow one find request from the client to the result.



Find products

```
db.products.find(  
  { category: "LAPTOP" }  
)
```

Parse and authorize

mongod checks the request and the user's rights.

Choose a plan

Use an index, or scan the whole collection.

Return BSON

Matching documents go back to the client.

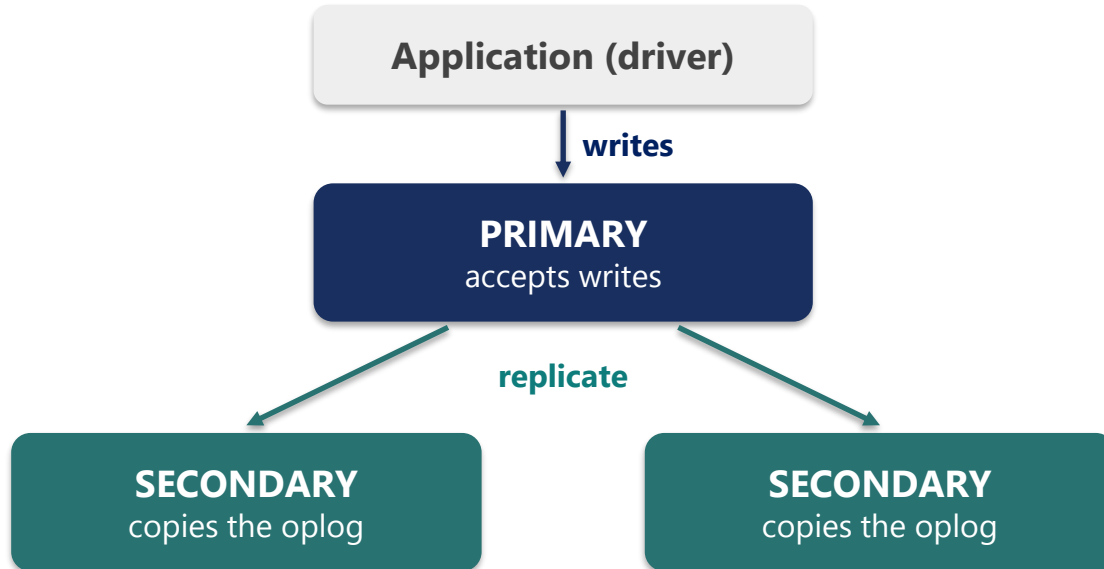
KEY TAKEAWAY The query planner's choice — index or collection scan — decides how much work each request does.

REPLICA SETS AND SHARDED CLUSTERS

Copies of the data for availability — and pieces of the data for scale.



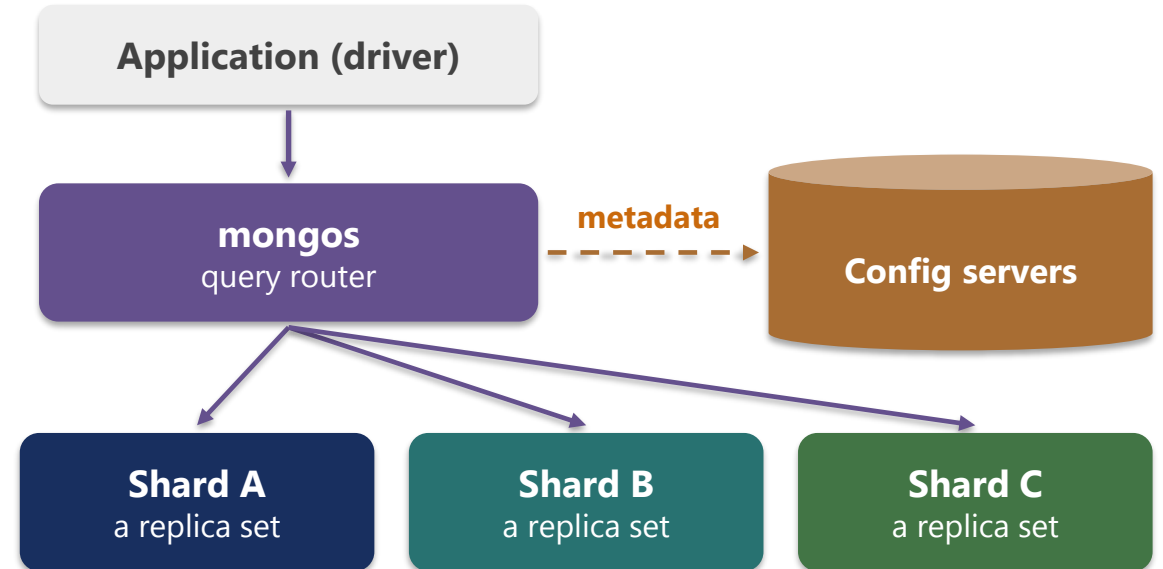
REPLICA SET: COPIES FOR AVAILABILITY



FAILOVER If the primary fails, the members elect a secondary as the new primary.



SHARDED CLUSTER: SPLIT DATA FOR SCALE

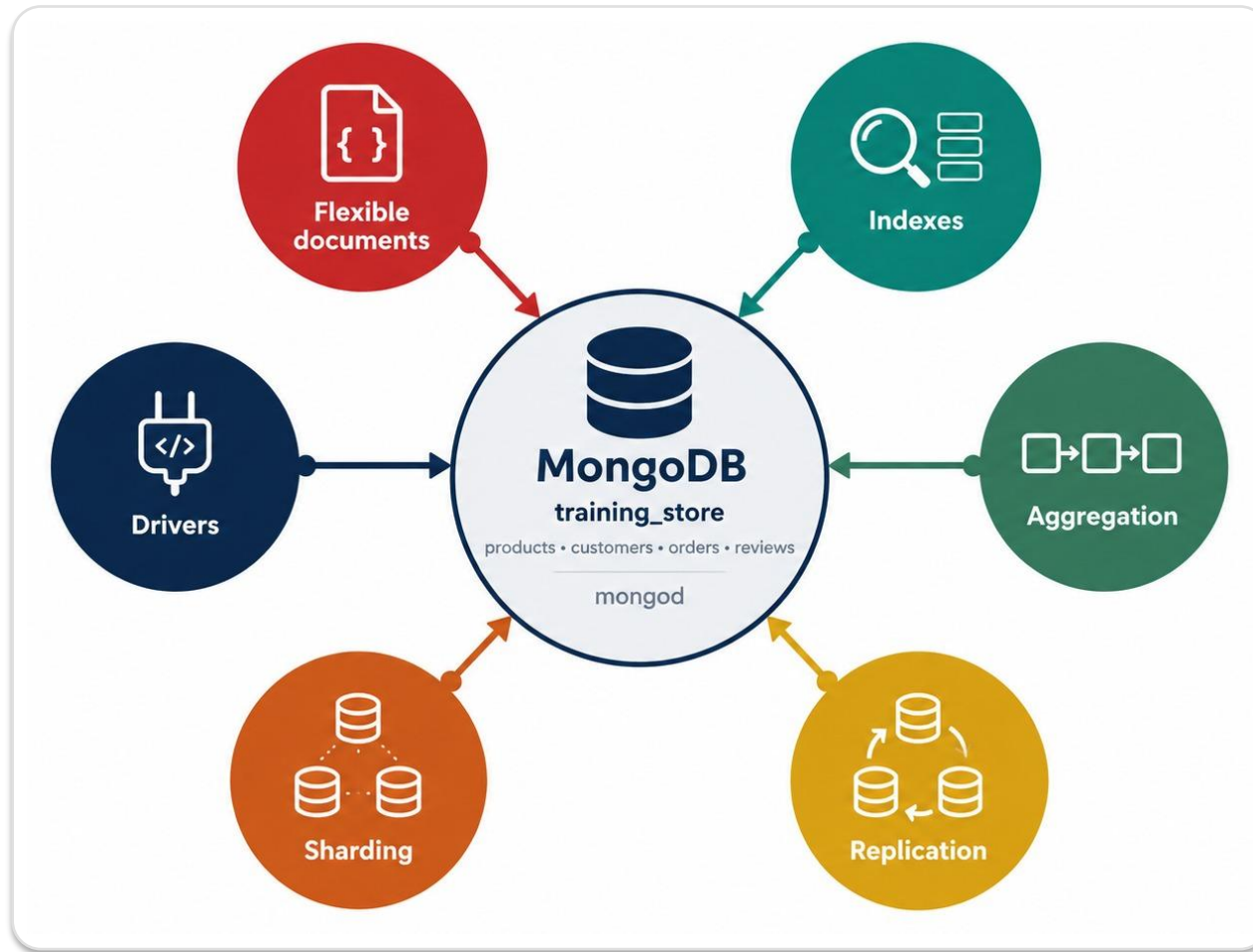


SHARD KEY Decides which shard stores each document — Module 7.

KEY TAKEAWAY Replica sets keep the data available through a failure; sharding lets it grow past one server.

CORE MONGODB FEATURES

Flexible documents, indexes, aggregation, replication, sharding and drivers.



Rich queries

Filter, sort, project and update — plus arrays, geospatial and text.

Indexes

Find documents without scanning the whole collection.

Aggregation

Pipelines of stages for reporting and analysis.

Rules when needed

Schema validation, atomic writes and multi-document transactions.

KEY TAKEAWAY MongoDB is more than flexible storage: it queries, indexes, aggregates, replicates and scales.

QUERIES, INDEXES AND AGGREGATION IN MONGOSH

Three commands on training_store — covered in depth on Days 2 and 3.

Filter and project

```
db.products.find(
  { category: "BOOK" },
  { name: 1, price: 1, _id: 0 }
)
```

find

Returns the two books — name and price only.

Index a common query

```
db.products.createIndex(
  { category: 1, price: 1 }
)
```

createIndex

Supports queries that filter on category and price.

Aggregate paid orders

```
db.orders.aggregate([
  { $match: {
    paymentStatus: "PAID" } },
  { $group: {
    _id: "$customerId",
    spent: { $sum: "$total" } } },
  { $sort: { spent: -1 } }
])
```

aggregate

Totals paid orders per customer; C101 comes first.

KEY TAKEAWAY One query language filters, indexes and aggregates — all on the same documents.

EXERCISE: PREDICT THE QUERY RESULT

Read a query and say what mongosh prints — before running it.

PRACTICE EXERCISE

5 MIN

SCENARIO

training_store has three shoes: Running Shoe 129.99, Trail Shoe 89.99 and Clearance Shoe 44.99.

```
db.products.find(
  { category: "SHOE",
    price: { $lt: 100 } },
  { name: 1, price: 1, _id: 0 }
).sort({ price: 1 })
```

YOUR TASKS

- 1 Say which shoes match the filter
- 2 Put them in the order sort() returns
- 3 List the fields each result shows
- 4 Check your answer on the instructor's screen

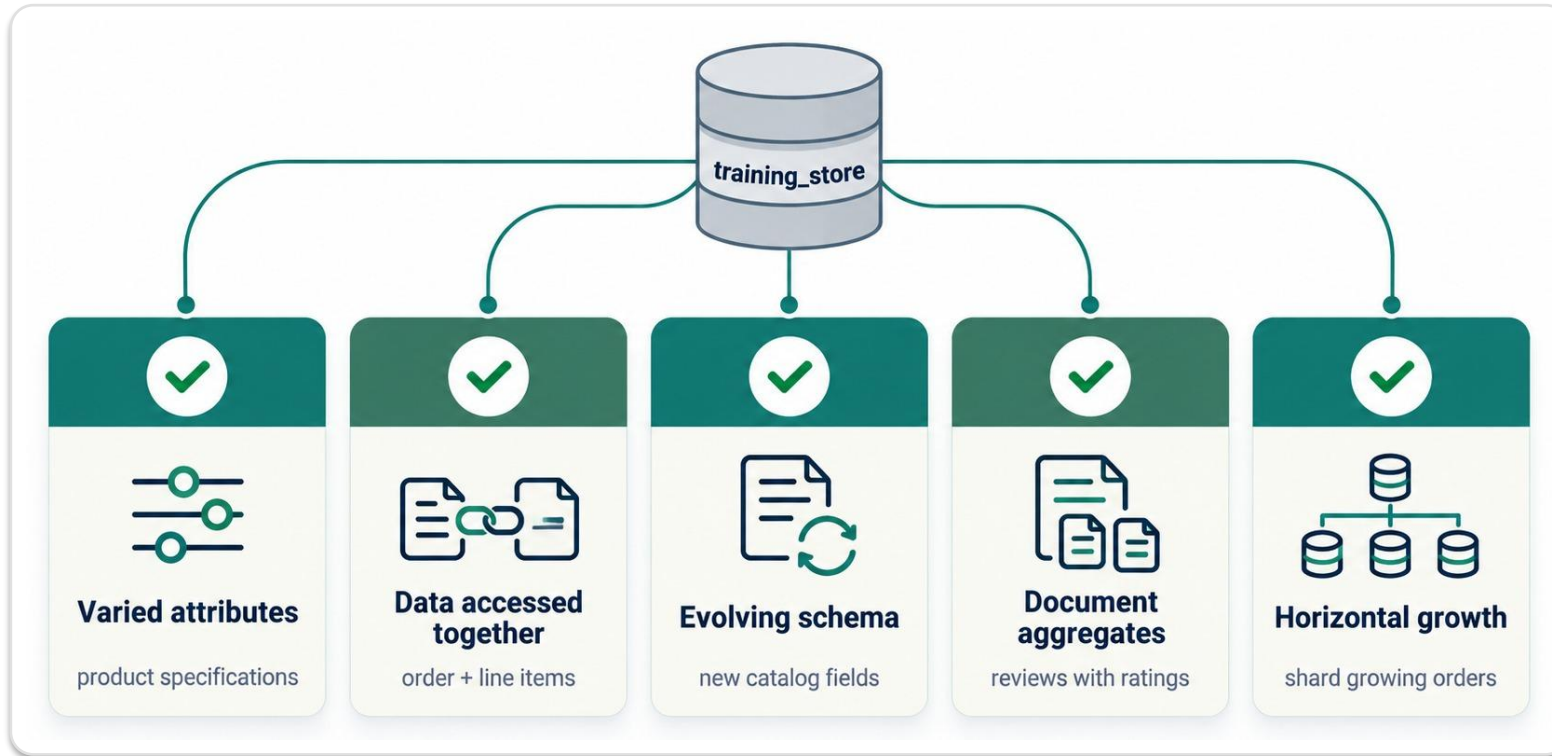
EXPECTED OUTCOME

- ✓ Clearance Shoe (44.99), then Trail Shoe (89.99)
- ✓ Running Shoe is excluded: 129.99 is not under 100
- ✓ Only name and price — _id is hidden
- ✓ Prices print as Decimal128 values

KEY TAKEAWAY Reading a query before running it is the fastest way to learn the query language.

WHEN TO USE MONGODB

Five signals that point to a document database.



Varied attributes

Products with category-specific specifications.

Accessed together

An order and its line items, read in one call.

Evolving schema

New catalog fields without a migration.

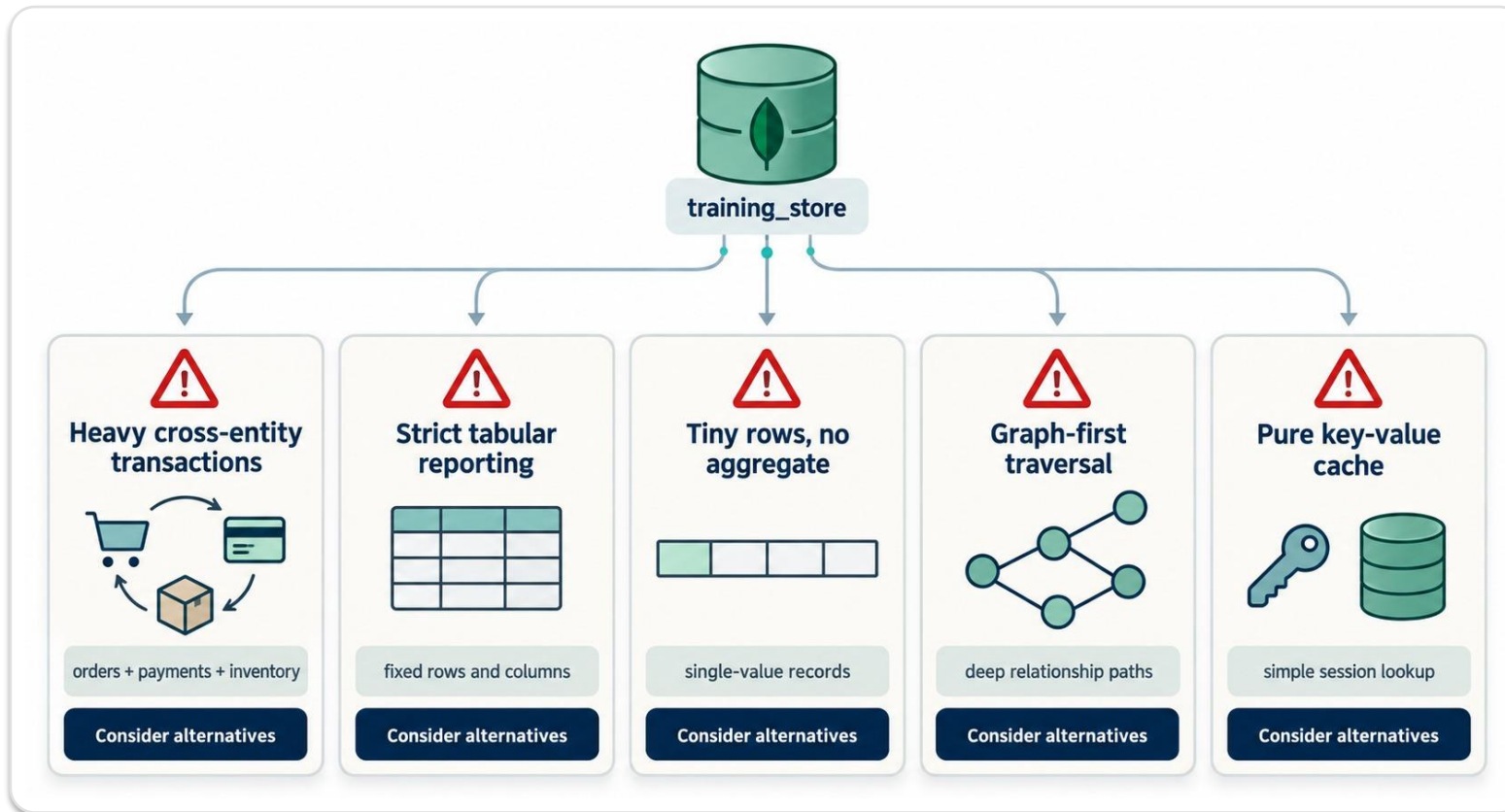
Horizontal growth

Shard a fast-growing orders collection.

KEY TAKEAWAY The more of these signals a workload shows, the better MongoDB fits.

WHEN MONGODB MAY NOT BE THE BEST CHOICE

Five workloads where another model may fit better.



Cross-entity transactions

Supported — but if most work spans many entities, relational may be simpler.

Strict tabular reporting

Fixed rows and columns suit relational tools.

Graph-first or pure cache

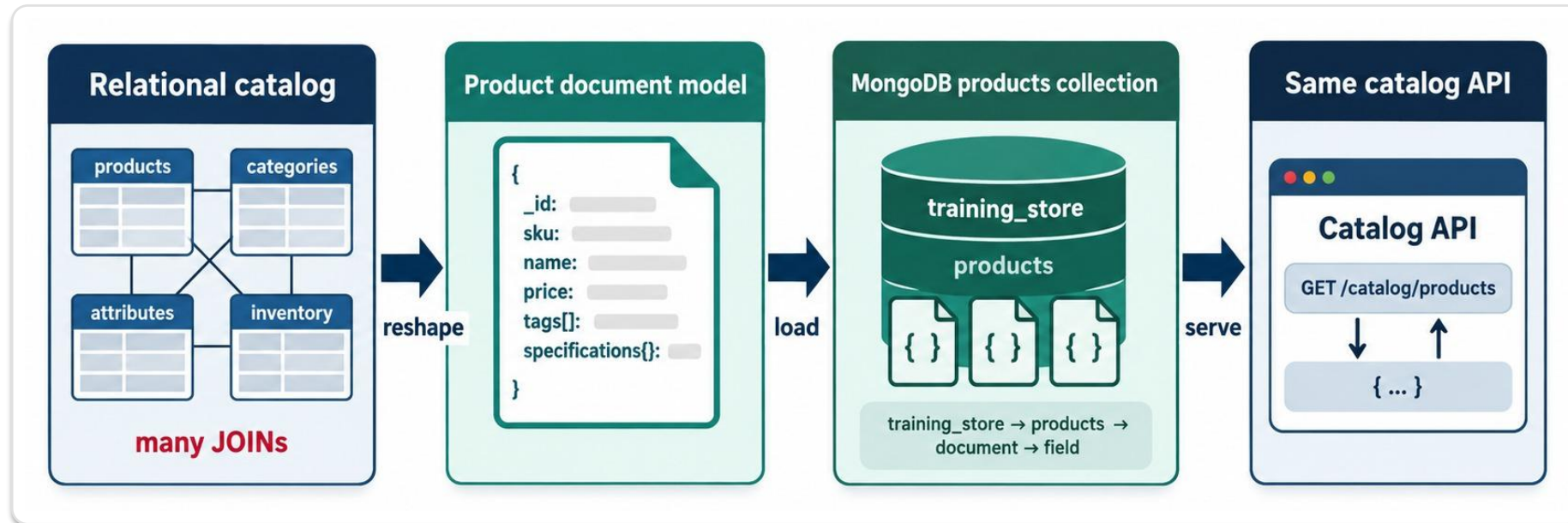
Deep traversal → graph; session lookup → key-value.

CONSIDER ALTERNATIVES Check the workload — don't rule MongoDB out.

KEY TAKEAWAY Knowing when not to use MongoDB is part of choosing it well.

SCENARIO: MODERNIZING A PRODUCT CATALOG

Reshape the data, load it, and serve the same API.



Before

products, categories, attributes and inventory tables — many joins.

Reshape and load

One product document: sku, name, price, tags, specifications.

Same API

GET /catalog/products keeps its contract; one read per product.

MODEL FIRST Reshape around reads — don't copy tables.

KEY TAKEAWAY Modernizing starts by reshaping the data around how it is read — not by copying the tables.

MISCONCEPTIONS AND ANTI-PATTERNS

Beliefs that lead first MongoDB designs into trouble.

MISCONCEPTION

REALITY

NoSQL means SQL is obsolete

Not Only SQL — relational still fits many workloads

Schemaless means no design

A flexible schema still needs a model — and can be validated

MongoDB has no transactions

Single-document writes are atomic; multi-document transactions exist

Copy each table into a collection

Model documents around how the app reads and writes

NoSQL is one technology

Four families with very different strengths

ANTI-PATTERNS TO AVOID

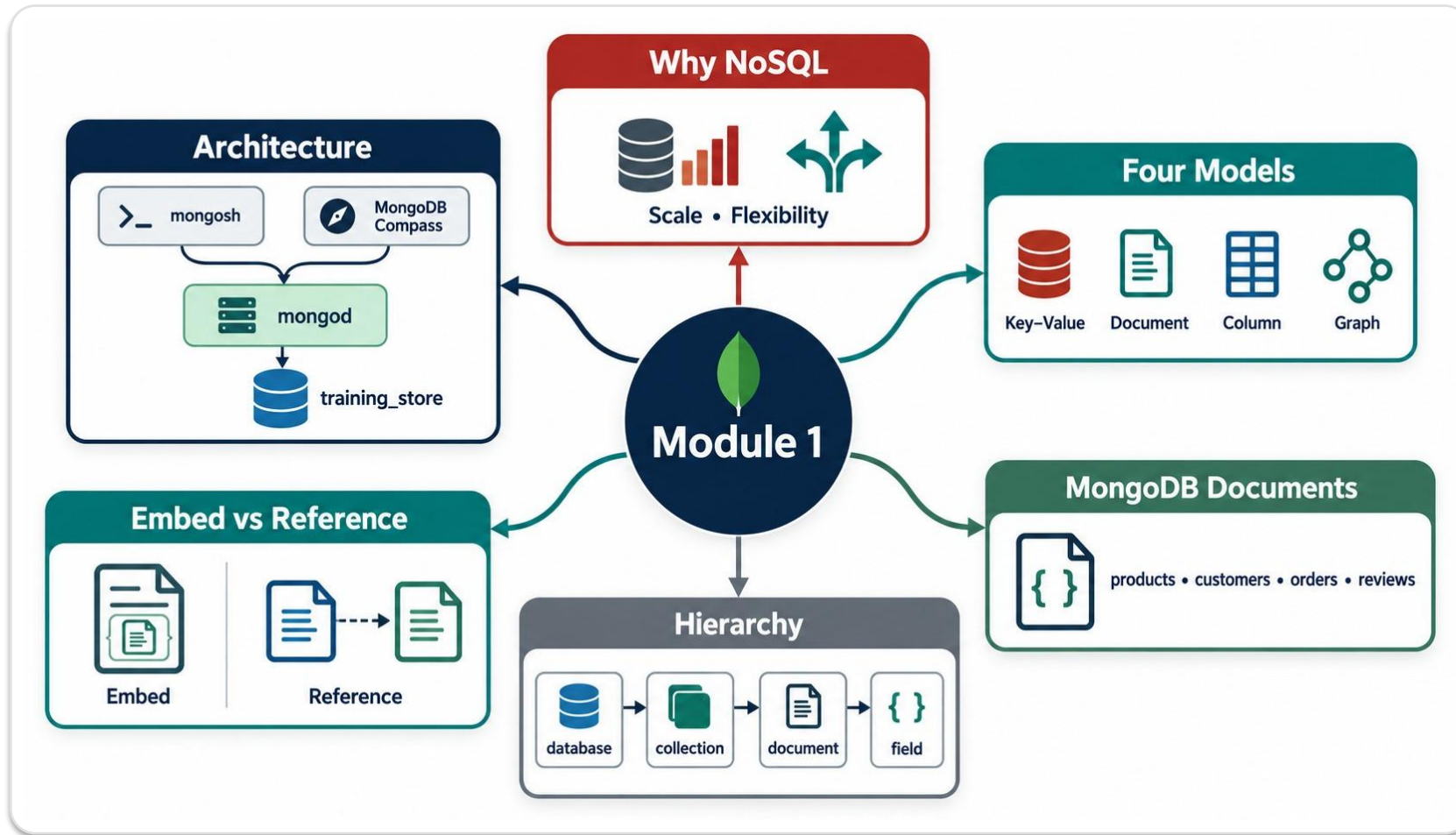
- ✗ One collection per relational table
- ✗ Unbounded arrays, e.g. every review inside its product
- ✗ Prices stored as strings
- ✗ No indexes for common queries
- ✗ A \$lookup join on every read

REMEMBER Store together what the application reads together.

KEY TAKEAWAY Most first-design problems come from treating MongoDB like tables with a new syntax.

MODULE 1 AT A GLANCE

Why NoSQL, four models, MongoDB documents and architecture — on one page.



KEY TAKEAWAY NoSQL is a family of models; MongoDB stores BSON documents shaped around how the application reads them.

KNOWLEDGE CHECK

Answer in your own words — then compare with a partner.

1 What does “Not Only SQL” communicate?

2 Name the four NoSQL model families.

3 Which model fits highly connected data?

4 What is MongoDB's equivalent of a table? Of a row?

5 Which field uniquely identifies a document?

6 What is BSON, and why not plain JSON?

7 Why might a product catalog fit a document database?

8 Does a flexible schema mean no schema design?

9 What do replica sets and sharding each provide?

10 Name one workload where MongoDB may not fit best.

EXERCISE 1.1 — CHOOSE THE APPROPRIATE DATA MODEL

Module 1 checkpoint A: match workloads to models, and justify each one.

OFFICIAL CHECKPOINT A

15 MIN

labs/day-01/exercises/exercise-1.1-choose-the-data-model.md

SCENARIO

Pick one starting model for each workload: relational, document, key-value, column-family or graph.

- A Online shopping sessions
- B Product catalog
- C Social recommendations
- D Banking ledger
- E Sensor platform
- F Content management

YOUR TASKS

- 1 Name each model's primary structure
- 2 Classify A–C with a one-line reason
- 3 Classify D–F the same way
- 4 Defend one controversial row to the class

SOLUTION (AFTER DEBRIEF)

- ✓ A key-value · B document
- ✓ C graph · D relational
- ✓ E column-family or time-series
- ✓ F document
- ✓ Every reason names an access pattern

KEY TAKEAWAY More than one technology may work — start from the access pattern, not popularity.

EXERCISE 1.2 — CONVERT RELATIONAL ROWS INTO A DOCUMENT

Module 1 checkpoint B: one order, one document.

OFFICIAL CHECKPOINT B

20 MIN

labs/day-01/exercises/exercise-1.2-rows-to-documents.md

SCENARIO

Order O5001 lives in three tables. Make it one document the application can read without joins.

Customer	C101	Aisha Khan	
		aisha@example.com	
Order	O5001	PLACED	110.00
Items	P10	Keyboard	2 × 25.00
	P22	Mouse	1 × 60.00

YOUR TASKS

- 1 Name the three tables and their join keys
- 2 Draft the order header: status and total
- 3 Embed the items and the customer; set `_id`
- 4 Decide what is a snapshot and what can go stale

EXPECTED OUTCOME

- ✓ One document with `_id`: "O5001"
- ✓ A nested customer and an items array
- ✓ Purchase-time price is a snapshot
- ✓ An embedded email can go stale
- ✓ Shaped for: get O5001 in one read

KEY TAKEAWAY Some duplication is intentional when it serves the access pattern and preserves history.

EXERCISE 1.3 — EXPLORE A MONGODB DATASET

Module 1 checkpoint C: navigate training_store and read real documents.

OFFICIAL CHECKPOINT C

25 MIN

labs/day-01/exercises/exercise-1.3-explore-a-mongodb-dataset.md

SCENARIO

The instructor has loaded training_store. No mongosh yet? Follow on screen.

```
show dbs
use training_store
show collections
db.products.findOne()
db.orders.findOne()
db.products.find().limit(2)
// a laptop and a book:
db.products.find(
  { sku: { $in: ["L100", "B300"] } })
```

YOUR TASKS

- 1 List the databases and collections
- 2 Read one product and one order
- 3 Label _id, scalars, objects, arrays, dates, numbers, Booleans
- 4 Compare two products' fields
- 5 Answer the observation sheet

EXPECTED OUTCOME

- ✓ customers, orders, products, reviews
- ✓ _id: the unique document identifier
- ✓ Order: shippingAddress object, items array
- ✓ Products differ by category attributes
- ✓ One row-versus-document difference

KEY TAKEAWAY A document looks like an application object: nested fields, arrays and real types in one record.

MODULE 1 SUMMARY AND WHAT'S NEXT

From why NoSQL exists to reading real MongoDB documents.

YOU CAN NOW

- ✓ Explain what NoSQL means and why it emerged
- ✓ Compare document, key-value, column-family and graph models
- ✓ Describe databases, collections, documents and BSON
- ✓ Explain mongod, replica sets and sharding
- ✓ Judge when MongoDB fits — and when it doesn't
- ✓ Read training_store documents in mongosh

NEXT: MODULE 2 — INSTALLATION AND SETUP

- 1 Local MongoDB or MongoDB Atlas?
- 2 How do I install and start mongod and mongosh?
- 3 How do I connect and load training_store?

BRING ALONG Your Exercise 1.1 table, your O5001 document and your Exercise 1.3 observation sheet.

KEY TAKEAWAY Module 2 gives you your own running MongoDB, so every later module can use training_store hands-on.